Nomor Port Unik

Untuk kepentingan kuliah ini, mahasiswa perlu memilih sebuah nomor dengan 4 sampai 5-digit angka yang akan digunakan sebagai nomor port unik yang nantinya dipakai untu *networking services*. Silakan rujuk kembali dokumen A01a untuk nomor port ini.

Deskripsi

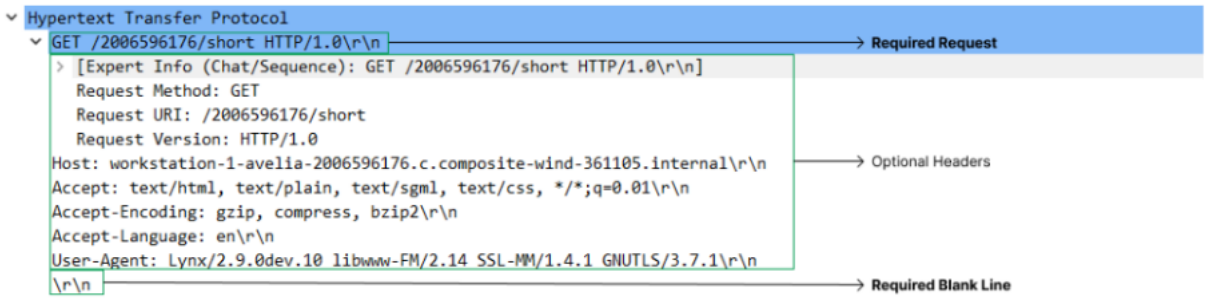
Hypertext Transfer Protocol (HTTP)

HTTP adalah protokol yang mendefinisikan komunikasi antar web browser (*client*) dan *sever*. *Client* dan *server* bertukar data menggunakan pesan HTTP. Terdapat dua jenis pesan, yaitu *request* dan *response*. *Request* dapat dikirim oleh *client* untuk melakkan suatu tindakan di *server*. *Server* dapat menjawab *request* dengan *response*.

Pesan HTTP terdiri dari informasi berbentuk teks ASCII, dan terdiri dari beberapa baris. Berikut ini adalah contoh bagaiman pesan HTTP di-encode dalam ASCII (sisi kiri). Anda dapat memeriksa tabel ASCII Anda dan Anda akan menemukan bahwa teks di sebelah kanan adalah representasi dari kode ASCII yang ada di sisi kiri.

```
73 11 47 45 54 20 2f 32  30 30 36 35 39 36 31 37   s·GET /2 00659617
36 2f 73 68 6f 72 74 20  48 54 54 50 2f 31 2e 30   6/short  HTTP/1.0
0d 0a 48 6f 73 74 3a 20  77 6f 72 6b 73 74 61 74   ··Host:  workstat
69 6f 6e 2d 31 2d 61 76  65 6c 69 61 2d 32 30 30   ion-1-av elia-200
36 35 39 36 31 37 36 2e  63 2e 63 6f 6d 70 6f 73   6596176. c.compos
69 74 65 2d 77 69 6e 64  2d 33 36 31 31 30 35 2e   ite-wind -361105.
69 6e 74 65 72 6e 61 6c  0d 0a 41 63 63 65 70 74   internal ··Accept
```

Pada HTTP/1.1, pesan dikirim secara terbuka. HTTP/2 membagi pesan ke dalam bebera *frame* HTTP. Perubahan ini memberikan optimisasi dan peningkatan *performance*.



Request dan *response* HTTP dimulai dengan beberapa baris *header*, masing-masing diakhiri CRLF (*carriage return-line feed*, atau "\r\n" dalam C-string). Baris pertama dari *request* (bagian *Required Request*) harus merupakan baris *request* atau *response* yang valid. Setelah baris pertama *request* ini dapat memberikan informasi yang berguna bagi *client* dan *server*, seperti browser apa yang sedang dipakai. Pada akhir baris-baris *header*, ada satu baris kosong (hanya terdiri dari CRLF, ditulis "\r\n" pada contoh). Bagian baris kosong ini diperlukan sesuai dengan spesifikasi HTTP. Untuk response HTTP, di bawah bagian tersebut terdapat *message body*. Bagian

ini bisa berisi HTML dan dapat juga kosong. Untuk detil spesifikasi HTTP lebih lanjut, Anda dapat membaca RFC 2616 dan RFC 7231.

Spesifikasi

Dalam tugas ini, mahasiswa ditugaskan untuk menerapkan teknik *socket programming* untuk mengimplementasikan HTTP pada program *client* dan *server* yang dibangun dari awal. Tujuannya adalah untuk menghasilkan sepasang client dan server yang fungsional dan dapat berkomunikasi sesuai HTTP/1.1. Pasangan ini dikatakan fungsional jika client dapat membuat request ke *server* dan *server* dapa menangani *request* tersebut menggunakan HTTP/1.1 dengan *header* yang ditentukan pada tugas ini.

Pada A03, mahasiswa membangun dua program independen: sebuah HTTP client dan sebuah HTTP server. Keduanya berkomunikasi melalui TCP. Berbeda dari penggunaan web framework atau package HTTP siap pakai, mahasiswa akan membentuk dan membaca pesan HTTP sendiri agar hubungan antara application-layer protocol dan TCP dapat diamati secara langsung.

HTTP/1.1 memiliki cakupan yang jauh lebih besar daripada yang dibutuhkan dalam tugas ini. Oleh karena itu, A03 menggunakan A03 HTTP/1.1 Profile: subset yang sengaja dibatasi dan didefinisikan pada dokumen ini. Implementasi hanya diwajibkan mendukung fitur yang tercantum dalam profile tersebut. Mahasiswa dapat menggunakan RFC 9110, RFC 9112, dan dokumentasi MDN sebagai referensi konseptual, tetapi tidak diharapkan mengimplementasikan seluruh HTTP/1.1.

Interoperability merupakan bagian inti dari tugas. Program client tidak cukup hanya dapat berbicara dengan server buatan mahasiswa sendiri, dan demikian pula sebaliknya. Kedua program diharapkan dapat dirancang agar bisa berkomunikasi dengan klien HTTP sungguhan.

Sebelum memulai, berikut adalah beberapa aturan umum dan/atau ekspektasi program yang akan dibuat:

1. Program client dan server harus diimplementasikan sebagai program independen, sehingga akan ada dua program yang berbeda.
2. Program harus ditulis dalam bahasa pemrograman Go untuk memberikan pengalaman yang konsisten bagi mahasiswa dan tim asisten.
3. Mahasiswa **harus menggunakan *template* kode yang telah disediakan** guna memastikan kontrak kode tetap terpenuhi dan menjamin penilaian bersifat deterministik.
4. **Tidak diizinkan untuk menggunakan library diluar bawaan dan/atau library yang terkait dengan HTTP** karena tugas ini mengharuskan mahasiswa untuk mengimplementasikan protokol komunikasi sendiri dari awal. Library bawaan seperti **fmt**, **net**, **os**, dan library untuk mashal tipe data seperti **encoding/xml** atau **encoding/json** serta library untuk kompresi data seperti **compress/gzip** atau **compress/flate** masih diperbolehkan. Jika ragu, silakan berkonsultasi dengan tim asisten mengenai library yang diizinkan.

5. Mahasiswa diharapkan untuk menggunakan TCP sebagai tipe socket karena akan mengimplementasikan HTTP/1.1.
6. Pekerjaan mahasiswa akan dinilai dengan bantuan "grader" menggunakan teknik unit testing. Oleh karena itu, mahasiswa diharuskan untuk menggunakan template kode yang diberikan atau setidaknya mahasiswa mengikuti instruksi yang diberikan dalam method skeleton yang disediakan.
7. Tim asisten telah menyediakan beberapa struct sebagai kesepakatan tentang bagaimana data harus disimpan dan digunakan dalam program, seperti HTTPRequest, HTTPResponse, Student, GreetResponse. Pastikan mengikuti spesifikasi dari struct tersebut.
8. Ingat bahwa HTTP menentukan penggunaan CRLF ("r\n") untuk memisahkan baris. Harap perhatikan hal ini, terutama ketika mengembangkan program di sistem *nix (macOS atau Linux) yang menggunakan LF ("\n") sebagai karakter default baris baru.
9. Raw syscall/socket API tidak diwajibkan. Implementasi yang menggunakan abstraction dari package net, termasuk net.Conn, merupakan pendekatan yang diharapkan.
10. Seluruh project harus dapat diperiksa dengan go build ./... dari root module pada environment Go 1.27.0, tanpa memerlukan binary hasil kompilasi atau third-party dependency di submission.

A03 HTTP/1.1 Profile

Tabel berikut mendefinisikan fitur HTTP yang wajib didukung. Fitur di luar tabel tidak menjadi requirement kecuali dinyatakan secara eksplisit pada bagian lain dokumen ini.

Fitur	Requirement A03
Transport	TCP
HTTP version	HTTP/1.1
Request method	GET
Line ending	CRLF (\r\n)
Akhir request header	Satu empty line: CRLF CRLF
Request body	Tidak digunakan
Response framing	Content-Length
Koneksi	Satu request dan satu response per koneksi
Connection persistence	Satu request/response per koneksi; client dan server memakai Connection: close
Required request header	Host wajib; Accept opsional pada server, tetapi wajib dikirim client A03; Accept-Encoding opsional
Required response header	Content-Type, Content-Length, dan Connection selalu ada; Content-Encoding hanya untuk gzip; Allow: GET pada 405
Header name	Nama header case-insensitive; urutan tidak signifikan; nilai Host mendukung host:port

Fitur	Requirement A03
Transfer-Encoding / chunked	Tidak didukung
Pipelining	Tidak didukung
TLS/HTTPS	Di luar cakupan A03

Nama header harus diproses tanpa membedakan huruf besar/kecil. Urutan header tidak signifikan; spasi horizontal di sekitar nilai dapat diabaikan. Header harus dipecah pada tanda titik dua pertama (nilai Host dapat mengandung tanda titik dua). Header tambahan yang tidak dikenal harus diabaikan dengan aman. Grader tidak mengirim header duplikat, folded/multiline header, Transfer-Encoding, atau HTTP message body. Server menulis nilai Content-Type persis text/html, application/json, application/xml, atau text/plain, tanpa parameter seperti charset.

Program Client

Client bertanggung jawab untuk membentuk request HTTP/1.1, membuka koneksi TCP ke target server, mengirim request, membaca response lengkap, melakukan decoding yang diperlukan, dan menampilkan hasil kepada pengguna. Detail interface command-line akan ditetapkan pada starter project.

- Client harus dapat menentukan target host/IP dan port server, URI yang diminta, content type yang diharapkan, serta pilihan content encoding sesuai interface starter.
- Request harus dibentuk dari data pada struct request melalui encoder yang disediakan pada template, bukan menggunakan package HTTP siap pakai.
- Client A03 wajib mengirim Host (berisi host dan port tujuan), Accept, serta Connection: close. Accept-Encoding hanya dikirim ketika pengguna memilih gzip; pilihan none/identity berarti header ini dihilangkan. Nilai identity/none adalah pilihan pada antarmuka program, bukan Content-Encoding pada response.
- Client harus membaca response berdasarkan framing HTTP, bukan berdasarkan asumsi bahwa seluruh response tersedia pada satu Read.
- Jika Content-Encoding: gzip ditemukan, ResponseDecoder harus melakukan decompression setelah memastikan jumlah byte body pada wire cocok dengan Content-Length. Data hasil decoding di struct adalah body yang sudah didekompresi, sedangkan Content-Length tetap menunjukkan jumlah byte body yang diterima di wire.
- Output client untuk self-testing harus cukup jelas untuk menunjukkan status response, Content-Type, Content-Encoding jika ada, Content-Length, serta isi body setelah decoding/decompression.
- Client harus dapat berkomunikasi dengan reference server pada grader, bukan hanya dengan server mahasiswa sendiri.

Program Server

Server bertanggung jawab untuk menerima koneksi TCP, membaca request lengkap, mengubah pesan menjadi struct request, menjalankan routing/handler, membentuk struct response, meng-encode response HTTP/1.1, mengirimkannya ke client, lalu menutup koneksi tersebut dengan aman.

- Server harus listen pada port unik mahasiswa dan pada interface yang dapat diakses dari instance Client. Listen hanya pada localhost tidak memenuhi requirement remote deployment.
- Request harus di-decode melalui decoder yang disediakan pada template. Handler dan encoder response yang ditandai untuk grader harus tetap digunakan sesuai interface starter.
- Server tidak boleh crash akibat request malformed, URI tidak dikenal, header tambahan, atau koneksi yang terputus sebelum pesan lengkap. I/O jaringan harus memiliki timeout terhingga; waktu persisnya tidak dinilai, tetapi server/client tidak boleh menunggu tanpa batas.
- Server harus kembali menerima koneksi berikutnya setelah satu request selesai. Concurrency tidak menjadi requirement.
- Server harus dapat berkomunikasi dengan reference client pada grader.

Endpoint GET /

Request GET / digunakan sebagai health/identity check. Server mengembalikan 200 OK dengan Content-Type: text/html dan body UTF-8 persis "Halo, dunia! Aku <NAMA_ANDA> sedang mengerjakan A03" (tanpa tanda kutip dan tanpa newline tambahan), dengan <NAMA_ANDA> diganti nama mahasiswa yang dikonfigurasi. Endpoint / tidak menggunakan Accept negotiation atau gzip. Routing hanya memakai path sebelum tanda tanya; query pada / diabaikan sehingga GET /?x=1 tetap menghasilkan respons ini.

Endpoint GET /greet/<NPM>

Server menangani GET /greet/<NPM> dan GET /greet/<NPM>?name=<str>. Untuk mempertahankan perilaku A03 sebelumnya, <NPM> pada URI harus sama dengan NPM mahasiswa yang dikonfigurasi di server. Jika berbeda, server mengembalikan 404 Not Found (body kosong), tanpa melakukan negosiasi representasi. Nilai NPM tetap harus diperiksa dari URI yang diterima, bukan diabaikan.

- <NPM> adalah 10 digit desimal yang sama dengan NPM pemilik submission. URI dengan format NPM yang tidak valid, suffix tambahan, atau path lain mendapat 404 Not Found.
- Parameter query name, jika ada dan tidak kosong, menjadi nilai Greeter. Jika name tidak ada atau kosong, Greeter bernilai nama mahasiswa yang dikonfigurasi. Test grader untuk name menggunakan satu parameter tunggal berisi karakter ASCII unreserved (huruf, angka, underscore, atau tanda hubung).

- Body respons berupa GreetResponse yang berisi Student dan Greeter. Student.Nama selalu bernilai nama mahasiswa yang dikonfigurasi; Student.Npm selalu bernilai NPM mahasiswa tersebut. Greeter bernilai parameter name apabila tidak kosong, atau Student.Nama sebagai default. Nama field/elemen: GreetResponse, Student, Nama, Npm, dan Greeter; serialisasi harus dapat dibaca kembali oleh encoding/json atau encoding/xml.
- Parameter query lain boleh diabaikan. Grader tidak menguji percent-decoding, duplicate name, karakter reserved/ber-spasi pada name, atau kombinasi query di luar pola yang disebutkan pada A03.
- Identitas server (nama dan NPM) boleh berupa konfigurasi/konstanta per mahasiswa sesuai starter project, tetapi routing tetap wajib membaca, memvalidasi, dan membandingkan NPM dari URI; pemeriksa dapat menguji NPM yang tidak sesuai dan mengharapkan 404.

Representation dan Content Negotiation

Endpoint /greet mendukung application/json dan application/xml. Gunakan struct Student {Nama, Npm} dan GreetResponse {Student, Greeter} yang didefinisikan starter. Untuk JSON, nama key mengikuti nama field tersebut; untuk XML, elemen root adalah GreetResponse dengan elemen Student/Nama/Npm dan Greeter. Urutan field JSON, whitespace, dan indentation tidak diperiksa secara byte-for-byte selama data dapat di-decode dan nilainya benar. Standard library encoding/json dan encoding/xml diperbolehkan.

A03 menggunakan grammar Accept yang disederhanakan, bukan algoritme negosiasi RFC 9110 secara penuh: satu atau lebih media type yang dipisah koma, dengan q-value opsional berbentuk ;q=0 sampai ;q=1 (maksimum tiga angka desimal). Whitespace di sekitar koma dan titik koma boleh ada. Grader hanya menguji grammar valid; media type duplikat, wildcard yang dicampur media type lain, dan parameter selain q berada di luar cakupan.

- q-value yang tidak dituliskan dianggap bernilai 1.0. q=0 berarti representation tersebut tidak dapat diterima.
- Jika beberapa supported representation diberikan, pilih representation dengan q-value positif tertinggi.
- Jika application/json dan application/xml memiliki q-value tertinggi yang sama, pilih application/json.
- Jika Accept tidak dikirim, kosong, atau hanya berisi */*, gunakan application/json sebagai default. Header Accept tetap wajib dikirim oleh program client A03 pada pengujian normal.
- Media type yang tidak didukung dapat diabaikan jika ada application/json atau application/xml dengan q-value positif yang didukung. Jika tidak ada yang cocok, balas 406 Not Acceptable.
- Jika header Accept diberikan tetapi tidak menghasilkan supported representation dengan q-value positif, server mengembalikan 406 Not Acceptable.

- Perbandingan q-value berlaku pada request /greet dengan NPM yang valid saja. Untuk request yang menghasilkan 404 atau 405, negosiasi tidak perlu dijalankan. Grader menggunakan q-value valid dan dapat mengubah whitespace di sekitar pemisah.

Content Encoding: gzip

A03 mendukung identity (tanpa kompresi) dan gzip pada respons sukses /greet. Deflate tidak diuji. Untuk menghindari dua algoritme q-value pada tugas pengantar Go, A03 sengaja tidak menggunakan bobot q pada Accept-Encoding. Kompresi dilakukan setelah JSON/XML selesai di-serialize, sebelum byte dikirim melalui TCP.

- Jika Accept-Encoding tidak dikirim, kosong, atau hanya berisi identity, server mengirim tanpa kompresi (identity) dan tidak menambahkan Content-Encoding.
- Jika token gzip ditemukan pada daftar nilai Accept-Encoding (dipisah koma, tanpa parameter q), gunakan gzip, termasuk ketika identity juga tercantum. Perbandingan token case-insensitive; whitespace di sekitar koma diabaikan.
- Jika hanya ada encoding yang tidak didukung (misalnya br) atau tidak ada token gzip, fallback A03 adalah identity. q-value, wildcard *, dan larangan identity seperti identity;q=0 tidak menjadi bagian input grader A03.
- Jika gzip digunakan, server wajib menambahkan Content-Encoding: gzip dan Content-Length harus menyatakan jumlah byte hasil kompresi, bukan panjang string atau panjang body sebelum kompresi. Jika identity digunakan, header Content-Encoding dihilangkan.
- Client mengenali Content-Encoding: gzip, mendekompresi response body sebelum menampilkan hasil, dan tetap melaporkan Content-Length dari response header sebagai jumlah byte yang diterima sebelum dekompresi.
- Compressed byte sequence tidak harus identik byte-for-byte dengan reference implementation; hasilnya harus merupakan gzip yang valid dan menghasilkan payload yang benar setelah decompression.

Error Handling dan Robustness

Seluruh respons error A03 (400/404/405/406) wajib berbentuk HTTP/1.1 valid: reason phrase standar sesuai status, Content-Type: text/plain, Content-Length: 0, Connection: close, tanpa Content-Encoding, dan body kosong. Khusus 405, sertakan Allow: GET sesuai RFC 9110. Content-Length adalah jumlah byte pada wire (octet), bukan jumlah karakter.

Status	Kondisi minimum yang wajib ditangani
400 Bad Request	Request line/header malformed, Host hilang/kosong, atau header >8192 byte. Jika peer sudah terputus sebelum request lengkap, server boleh langsung menutup koneksi tanpa respons.
404 Not Found	Path selain / atau /greet/<NPM_ANDA> (termasuk NPM tidak cocok dan pola URI tidak valid).

Status	Kondisi minimum yang wajib ditangani
405 Method Not Allowed	Method selain GET setelah sintaks/Host valid; header tambahan Allow: GET wajib dikirim.
406 Not Acceptable	Hanya untuk /greet valid: Accept eksplisit tidak menyediakan JSON/XML dengan q-value positif.

Jika peer memutus koneksi sebelum pesan lengkap dan respons tidak dapat dikirim, server cukup membersihkan resource dan melanjutkan menerima koneksi lain, tanpa panic. Client harus melaporkan error jika header/body response terpotong. Untuk request yang cukup lengkap untuk dievaluasi, urutan keputusan status ialah: validasi sintaks dan Host (400), method (405), route/NPM (404), lalu Accept (406); jika lolos semua, 200.

Matriks Perilaku Normatif

Matriks ini adalah ringkasan kontrak yang akan menjadi acuan bagi starter, reference peer, dan grader. Gunakan <NPM_ANDA> untuk NPM yang dikonfigurasi di server; <NPM_LAIN> untuk 10 digit berbeda. Semua kasus diasumsikan memakai request HTTP/1.1 dengan Host valid.

Permintaan / kondisi	Respons yang wajib dihasilkan
GET /	200; text/html; pesan Halo, dunia!; identity
GET /greet/<NPM_ANDA>; Accept tidak ada atau */*	200; application/json; Greeter = nama mahasiswa
GET /greet/<NPM_ANDA>?name=Rani; Accept: application/xml	200; application/xml; Greeter = Rani
GET /greet/<NPM_ANDA>; Accept: application/xml;q=0.8, application/json;q=0.9	200; JSON (q tertinggi)
GET /greet/<NPM_ANDA>; Accept: application/xml;q=0.9, application/json;q=0.9	200; JSON (tie-break)
GET /greet/<NPM_ANDA>; Accept: application/json; Accept-Encoding: gzip	200; JSON terkompresi; Content-Encoding: gzip; Content-Length = byte terkompresi
GET /greet/<NPM_ANDA>; Accept-Encoding: br	200; JSON tanpa kompresi; tanpa Content-Encoding
GET /greet/<NPM_LAIN>	404; body kosong
GET /lainnya	404; body kosong
POST /	405; Allow: GET; body kosong
GET /greet/<NPM_ANDA>; Accept: text/plain	406; body kosong
Request malformed atau Host tidak tersedia	400; body kosong, jika koneksi masih dapat digunakan

Untuk seluruh baris respons: status line selalu HTTP/1.1, header Content-Type/Content-Length/Connection selalu dikirim, dan koneksi ditutup setelah respons. Matrik ini tidak mewajibkan byte gzip atau urutan header identik dengan reference peer.

Kontrak Interface Starter/Grader

Starter yang diterbitkan bersama A03 harus mempertahankan antarmuka berikut. Nama dan signature final disediakan secara eksplisit dalam source template yang dirilis bersamaan dengan tugas. Decoder/encoder yang menerima input tidak valid harus mengembalikan error idiomatis Go, bukan panic atau struct kosong yang menyembunyikan kegagalan.

Fungsi	Tanggung jawab dan hasil
RequestEncoder	HttpRequest → byte HTTP request + error; menyusun request line/header/CRLF.
RequestDecoder	byte request lengkap → HttpRequest + error; memeriksa request line/header.
ResponseEncoder	HttpResponse → byte HTTP response + error; termasuk gzip dan Content-Length wire.
ResponseDecoder	byte response lengkap → HttpResponse + error; termasuk validasi Content-Length dan dekompresi gzip.
HandleRequest	HttpRequest → HttpResponse; menangani route, NPM, Accept, dan status.
Fetch	HttpRequest + net.Conn → HttpResponse + error; mengirim request dan membaca respons lengkap.
HandleConnection	net.Conn → error; membaca request lengkap, menggunakan decoder/handler/encoder, dan menulis response.

Struct HttpResponse menggunakan StatusCode numerik (int), bukan string, dan menyimpan body sebagai []byte. Struct HttpRequest menyimpan method, URI, versi, dan header. Nama field persis serta konfigurasi identitas/server ditetapkan pada starter. Hasil ResponseDecoder menampilkan body yang telah didekompresi jika gzip; header Content-Length tetap merekam panjang body saat dikirim. RequestEncoder dan ResponseEncoder selalu menulis HTTP/1.1 pada start line; Version kosong diperlakukan sebagai HTTP/1.1, sedangkan Version lain menghasilkan error. Method yang valid secara sintaksis adalah token RFC 9110, yaitu satu atau lebih huruf, angka, atau karakter !#$%&'*+-.^_`|~. Method kosong atau bukan token menghasilkan error pada RequestEncoder dan 400 pada HandleRequest.

Interoperability dan Reference Peer

Automated grader memuat reference peer yang dapat bertindak sebagai reference server maupun reference client. Reference peer digunakan untuk memisahkan kesalahan client dari kesalahan server dan untuk memastikan implementasi mengikuti protocol contract A03, bukan sekadar kesepakatan tidak sengaja antara dua program mahasiswa sendiri.

Alur Pengerjaan yang Disarankan

A03 sebaiknya dikerjakan secara bertahap. Menambahkan semua fitur sekaligus akan membuat debugging jauh lebih sulit, terutama karena mahasiswa juga sedang mempelajari Go.

- Tahap 1 - Pastikan starter project Go 1.27.0 dapat dibangun dan kedua program dapat dijalankan.
- Tahap 2 - Bangun koneksi TCP dasar dan selesaikan GET / tanpa negotiation atau compression.

- Tahap 3 - Implementasikan framing, request/response encoder-decoder, dan endpoint /greet.
- Tahap 4 - Tambahkan JSON/XML dan aturan Accept/q-value.
- Tahap 5 - Tambahkan gzip dan perhitungan Content-Length setelah compression.
- Tahap 6 - Jalankan seluruh test pada starter setelah setiap t.Skip dihapus, tambahkan test sendiri, lalu uji server dengan request mentah melalui nc, termasuk error cases.
- Tahap 7 - Jalankan pasangan client-server pada dua instance AWS dan kumpulkan evidence untuk laporan.

Self-Testing dan Laporan

Self-testing tetap wajib agar mahasiswa membangun, menjalankan, mengamati, dan mendokumentasikan program. T00 memperlihatkan setup Go dan proses build. T01-T06 harus memperlihatkan student-client pada instance AWS Client berkomunikasi dengan student-server pada instance AWS Server yang sama. T07 memperlihatkan hasil go test -v ./... setelah seluruh t.Skip pada test starter dihapus. T08 memperlihatkan request mentah yang dikirim dari instance Client ke student-server menggunakan nc. Identitas instance/IP dan port harus tercantum pada laporan.

Untuk tiap test, lampirkan screenshot terminal dengan invocation, output/status, serta konteks host yang cukup untuk memverifikasi eksekusi. Tulis 2-4 kalimat tentang input, respons teramati, dan aturan A03 yang relevan. Untuk T07, cantumkan keluaran go test apa adanya (PASS maupun FAIL) dan jelaskan hasilnya; penilaian kebenaran program tetap dilakukan pada automated grader. Jangan menyunting keluaran untuk menyembunyikan kegagalan.

Kode	Evidence yang wajib ditunjukkan	Poin
T00	Go 1.27.0 terpasang pada environment dan project berhasil dibangun.	2
T01	Remote GET / via dua AWS instance menghasilkan 200 OK, text/html, dan identity body yang benar.	2
T02	/greet menghasilkan application/json dengan data yang benar.	3
T03	/greet menghasilkan application/xml dengan data yang benar.	3
T04	Satu request dengan lebih dari satu Accept value memilih representation berdasarkan q-value.	2
T05	/greet menggunakan gzip; response header dan hasil body setelah decompression terlihat benar.	2
T06	Request ke URI /lainnya menghasilkan 404 Not Found tanpa crash.	2
T07	Jalankan go test -v ./... setelah seluruh t.Skip pada test starter dihapus; dokumentasikan keluaran apa adanya (PASS maupun FAIL).	2

Kode	Evidence yang wajib ditunjukkan	Poin
T08	Dari instance Client, kirim request mentah ke student server dengan nc: POST / menghasilkan 405 dengan Allow: GET, request malformed menghasilkan 400, lalu GET / tetap berhasil.	2

Total komponen Self-Testing dan Laporan: 20 poin.

Skor T00-T08 mengukur pelaksanaan pengujian, kelengkapan bukti, dan penjelasan yang sesuai dengan hasil nyata; bukti FAIL yang dilaporkan jujur tetap dapat dinilai sebagai aktivitas self-testing, sedangkan correctness ditentukan grader otomatis. Tim Pengajar menyediakan starter beserta petunjuk penggunaannya sebelum tugas ini dinyatakan dirilis; reference peer hanya digunakan pada automated grader.

Automated Grading

Source code akan dinilai menggunakan grader yang menjalankan unit test dan integration test. Mahasiswa boleh menambahkan helper selama interface yang ditandai untuk grader tidak diubah. Grader tidak hanya membandingkan output contoh; grader dapat menggunakan input berbeda dan pola TCP delivery berbeda selama masih berada dalam A03 HTTP/1.1 Profile.

Komponen	Poin
Build dan interface contract	5
Request Encoder	10
Request Decoder	10
Response Encoder	10
Response Decoder	10
Routing dan application semantics (/ dan /greet)	10
JSON/XML dan simplified content negotiation	10
gzip content encoding	5
TCP stream handling, error handling, dan robustness	10

Total komponen Automated Grading: 80 poin. Nilai akhir A03 adalah jumlah komponen Automated Grading (80 poin) dan Self-Testing/Laporan (20 poin), sebelum pinalti lain yang berlaku berdasarkan bagian Peraturan.